PaperLens: Evidence-Grounded, Reproducibility-Aware Analysis of Research Papers with a Free-Tier Language Model

Author One, Author Two, Author Three, Author Four
Department of Computer Engineering
Sardar Patel Institute of Technology, Mumbai, India
{first.last}@spit.ac.in

***Abstract*—Large language models (LLMs) can summarise a research paper in seconds, but the facts they report come without evidence. For research, an invented number is worse than none. We present *PaperLens*, a system that takes a paper (PDF or link) and returns a structured breakdown in which every extracted item carries the verbatim sentence it came from. Python locates that sentence in the PDF and highlights it, and a second check judges whether it supports the item. On top of this grounded extraction, PaperLens scores reproducibility against a 21-item checklist, verifies numeric claims against the paper's own results tables, discovers the paper's code repository and compares its settings with the paper by static analysis, and offers page-cited chat, a notation glossary, a citation graph, a draft review and a multi-paper comparison. The design follows one rule: *the LLM extracts and Python decides*. Every score, delta and verdict is computed deterministically, and the whole system runs on the free tier of a commercial LLM API. On a 10-paper gold set, extraction reaches a core-field F1 of 0.72 with precision between 0.79 and 1.00 per field. Every extracted hyperparameter whose name matched the gold had the correct value, and 80% of extracted items are verified against the paper. Requiring a located quote lowers the hallucination rate from 10% to 2%, and adding the support check lowers it to 0%, at a cost in recall that we quantify. We release the system, the evaluation package and an annotation guide.**

***Index Terms*—scientific information extraction, grounding, attribution, reproducibility, large language models, document parsing, claim verification**

I. Introduction

Reading a research paper well takes one to three hours, and much of that time is spent finding the same facts: the datasets, baselines, metrics, hyperparameters, compute and limitations. Judging whether the work can be reproduced is harder still. Surveys of the AI literature found that no sampled paper documented every factor needed for reproduction [1]. An attempt to re-implement 255 papers without their code succeeded only about two times in three [2]. Released code does not settle the question either, because it often uses settings other than those reported [3], [4].

LLMs are an obvious aid, but their output is fluent and unattributed, and they hallucinate facts [5]. Attribution research shows that generated statements must be checked against an identified source to be trusted [6], [7]. For scientific use, where a wrong learning rate silently ruins a replication, unverifiable output is of little value.

We present *PaperLens*, built around a single principle: **the LLM extracts, and Python decides**. The LLM's only job is to read text and return schema-validated items, each with a verbatim quote. Deterministic code does everything else. It locates every quote in the PDF, scores reproducibility, matches claims to table cells and recomputes their deltas, compares the paper's hyperparameters with the code, and draws every verdict. The system runs on the free tier of the Gemini API [8], so every design choice also minimises the number of LLM calls.

Our contributions are:

1) An end-to-end system that parses a paper with layout coordinates and extracts a fixed schema. Every item carries evidence that is located in the PDF and checked for support, and items that cannot be verified are labelled rather than dropped.
2) Three deterministic checks built on the extraction: a reproducibility scorecard, a claim verifier that recomputes claimed differences from parsed tables, and a static paper-vs-code comparison that never executes repository code.
3) An evaluation package with a documented matching scheme, a gold set over ten widely used ML papers, and three ablations that need no LLM calls: evidence policy, parser and retrieval.
4) An honest assessment. Precision is high on every core field, the main weakness is recall on hyperparameters and tasks, and our reference annotations were produced with LLM assistance.

II. Related Work

A. Scientific information extraction

Entity and relation extraction from papers has been studied with dedicated datasets: SciERC for entities, relations and coreference [9], SciREX for document-level task/dataset/metric/method relations [10], and the leaderboard-construction work of Hou et al. [11]. AxCell extracts results from machine-learning papers, using table segmentation [12]. These systems rely on fine-tuned encoders such as SciBERT [13]. Dagdelen et al. fine-tune LLMs for joint named-entity and relation extraction that outputs structured JSON

for materials science [14]. PaperLens differs in two ways. It uses a general-purpose LLM without fine-tuning, and it makes evidence a first-class output that is checked against the source rather than an optional by-product.

B. Parsing scientific PDFs

GROBID turns PDFs into structured TEI with sections, references and coordinates [15], and it underlies corpora such as S2ORC [16]. Nougat converts page images to markup with a vision transformer [17], and Docling combines layout analysis with table structure recognition [18]. Table structure recognition has its own benchmarks [19], [20]. We found GROBID's table grids to be unreliable on our papers, so PaperLens rebuilds each grid from the PDF's word positions and uses GROBID for structure, sentences and citations (Section IV-A).

C. Grounded generation and attribution

Retrieval-augmented generation conditions an LLM on retrieved passages [21]. ALCE evaluates generated text whose statements cite their sources [7], and the AIS framework defines when a statement is attributable to an identified source [6]. FActScore breaks text into atomic facts and checks each against a knowledge source [22], while SelfCheckGPT detects hallucinations without external resources [23]. Long contexts are used unevenly by LLMs [24], which motivates our per-section extraction. PaperLens applies the attribution idea at the level of single extracted items and enforces it in code. A quote is accepted only if a fuzzy string match [25] finds it in the parsed paper.

D. Question answering and reading assistants

Qasper anchors information-seeking questions in research papers [26]. PaperQA [27] and OpenScholar [28] answer scientific questions with citations over large literature collections. Reading interfaces augment the paper itself. ScholarPhi shows position-sensitive definitions of terms and symbols [29], building on document-level definition detection [30], and the Semantic Reader project generalises these ideas [31]. TLDR generates extreme summaries of papers [32]. PaperLens works on a single paper. Its chat cites pages of that paper, and its notation glossary grounds each symbol's meaning in a sentence, in the spirit of ScholarPhi.

E. Scientific claim verification

SciFact verifies claims against abstracts [33]. TabFact [34] and SciTab [35] verify claims against tables, and SciTab shows that compositional reasoning over scientific tables remains hard for LLMs. We avoid asking the LLM for verdicts. It copies a claim's numbers, and Python finds the cells and recomputes the claimed difference.

F. Reproducibility

Beyond the surveys above [1], [2], the NeurIPS 2019 reproducibility programme introduced a checklist, a code policy and a reproducibility challenge [3]. Dodge et al. argue for reporting the expected validation performance as a function of the tuning budget [36], Bouthillier et al. separate methods reproducibility from inferential reproducibility [37], and Kapoor and Narayanan document data leakage across ML-based science [38]. Recent benchmarks ask agents to set up and run research code (SUPER [39], CORE-Bench [40]) or to replicate papers from scratch (PaperBench [41]). PaperLens takes the opposite and cheaper route. It never executes code, and it reports what a reproduction would need and where the paper and its code disagree.

TABLE I
POSITIONING AGAINST REPRESENTATIVE PRIOR WORK (✓ = CORE FEATURE, (✓) = PARTIAL OR INDIRECT, – = NOT A GOAL).

	Schema IE	Item evidence	Repro. check	Claims vs tables	Code check	Runs code
SciREX, AxCell [10], [12]	✓	–	–	(✓)	–	–
PaperQA, OpenScholar [27], [28]	–	✓	–	–	–	–
ScholarPhi [29]	–	✓	–	–	–	–
SciTab [35]	–	–	–	✓	–	–
CORE-Bench, PaperBench [40], [41]	–	–	✓	–	(✓)	✓
LLM reviewers [43]	–	–	(✓)	–	–	–
PaperLens	✓	✓	✓	✓	✓	–

G. LLM-assisted reviewing

Yuan et al. found generated reviews broad in coverage but less constructive and less factual than human ones [42]. Liang et al. report that GPT-4 feedback overlaps with human reviewers about as much as two humans overlap [43], and Zhou et al. find LLM reviewers unreliable for scoring [44]. PaperLens therefore offers a draft review without a score, and it lists facts established in Python separately from the LLM's opinions.

III. System Overview

Fig. 1 shows the architecture. A Next.js interface uploads a paper or a link (arXiv, DOI, OpenReview, ACL Anthology, a direct PDF URL). A FastAPI service stores it and queues a job, and a Celery worker runs the pipeline, one paper at a time, writing to PostgreSQL. Progress is streamed to the browser with server-sent events. Every LLM call goes through a single gateway. The gateway caches responses by (input, prompt version, model), rate-limits per model, retries with exponential backoff, switches to a backup model when one is overloaded, validates the output against a Pydantic schema with one repair attempt, and logs the provider, model and prompt version. Model names live only in configuration. Prompts are versioned templates, and results are keyed by (paper, stage, prompt version, model), so every stage is idempotent and a re-run of a cached paper costs nothing.

Table II lists the pipeline. Four of its stages make no LLM call at all, and extraction with verification uses about six calls per paper.

IV. Method

A. Parsing and table reconstruction

GROBID provides sections, sentences with coordinates, references and resolved in-text citations. All coordinates are

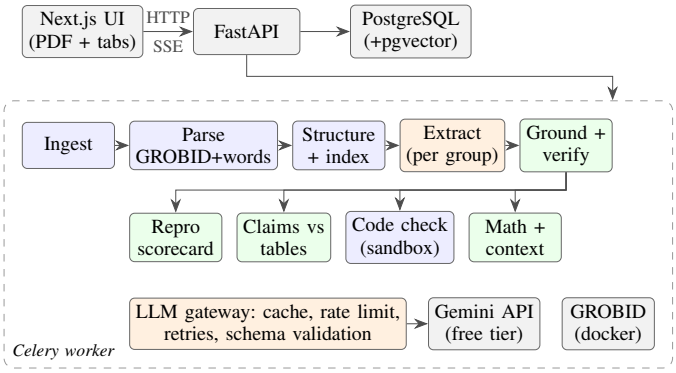


Fig. 1. Architecture. Orange stages call the LLM, blue stages are pure Python, and green stages make at most one LLM call whose output Python checks. The gateway is the only path to the LLM.

TABLE II
PIPELINE STAGES AND THEIR USE OF THE LLM.

Stage	LLM	What it does
Parse	none	GROBID TEI + PyMuPDF words → blocks with page boxes; table grids from word positions
Structure	none	section roles, paragraph chunks, BM25 + vector index
Extract	1/group	schema items, each with value, verbatim quote, chunk id
Verify	batched	quote located (fuzzy $\geq$ 90); support judged; unfound kept as *unlocated*
Repro	1	21-item checklist: rules first, LLM for the rest; Python score
Claims	1	LLM copies numbers; Python matches cells, recomputes, decides
Results	none	typed results tables, best value per column
Code	none	repository ranking, hardened clone, static scan, comparison
Math	1	LaTeX equations, grounded symbol glossary
Context	1	key references, citation graph, grounded novelty relations
Chat	1/turn	hybrid retrieval, cited answer or refusal
Review	1	draft review; findings supplied by Python; no score

kept in PDF points, so any span can be highlighted later. Its table grids, however, were often wrong or placed on the wrong page. PaperLens therefore locates each table by its own caption ("Table 3:") and rebuilds the grid from the PDF's word positions. Columns come from whitespace gaps, wrapped cells are merged, bold cells are detected, and values such as 24.3 ± 0.2 are parsed. A grid that fails a sanity check is dropped, because no grid is better than a wrong grid. If GROBID is unavailable, a PyMuPDF-only parser takes over.

B. Grounded extraction

Sections are grouped (front matter, method, experiments, closing), and each group is sent in one call with a JSON schema. Paragraphs are labelled with their chunk ids. For every item the LLM returns a value, a verbatim quote and the chunk id the quote comes from. Python then locates each quote q. It first tries the cited chunk c and then the whole paper D, using the partial-ratio similarity $s(\cdot,\cdot)$ of the normalised strings, which is a Levenshtein-based measure [25]:

$$\hat{b}(q) = \arg\max_{b \in \{c\} \cup D} s(q, b), \qquad \text{located} \iff s\big(q, \hat{b}(q)\big) \geq \tau, \tag{1}$$

with $\tau = 90$. A located quote is mapped to sentence boxes for highlighting. A table quote is a row written as the prompt showed it ("cell | cell"), and it highlights that row. A single batched call then labels each located quote as *verified* (it supports the item), *partial* or *unsupported*. A quote that is not found is labelled *unlocated*. Such items are kept and badged, never silently dropped, so the user sees them and the evaluation counts them.

C. Reproducibility scorecard

A 21-item checklist, derived from the NeurIPS checklist [3], is organised into five groups g with weights W_g. The groups are Code (0.25), Data (0.20), Experimental setup (0.25), Compute (0.10) and Statistical reporting (0.20). Rules decide an item first, using regular expressions, the document structure and GitHub metadata. One LLM call judges the remaining items, and each judgement must quote the paper. An LLM "present" whose quote is not confirmed is capped at "partial". With points $p(\text{present}) = 1$, $p(\text{partial}) = 0.5$ and $p(\text{missing}) = 0$, and item weights w_i over the items S_g of group g that are scored (not unknown or not applicable), the score is

$$\text{score} = 100 \cdot \frac{\sum_g W_g \, \sigma_g}{\sum_g W_g}, \qquad \sigma_g = \frac{\sum_{i \in S_g} w_i \, p(s_i)}{\sum_{i \in S_g} w_i}, \tag{2}$$

where the outer sums run over groups with at least one scored item.

D. Claim verification

For each quantitative claim, the LLM copies the method, baseline, metric, dataset, the claimed value or difference δ^*, and its unit and comparator, exactly as written. Python finds the candidate cells x_m (method) and x_b (baseline) in the parsed results tables, recomputes $\hat{\delta} = x_m - x_b$ (or a ratio for factor claims) and decides. The claim is *supported* if $|\hat{\delta} - \delta^*| \leq \epsilon$, with $\epsilon = 0.15$ absolute, set in configuration. *Contradicted* requires exactly one unambiguous pair of cells. Ambiguous or missing matches give *unsupported*, so a table the parser got wrong cannot make a paper look wrong.

E. Paper vs code

Repository links are ranked from the paper text, telling the authors' own code apart from libraries the work is "based on". The repository is cloned shallowly, with hooks, symlinks and local protocols disabled and a size cap. A parse-only scanner then runs inside a sandbox container and reads Python ASTs, YAML, JSON, TOML and README flags. It never executes repository code. Assignments under inference-only branches are skipped. Hyperparameter names are aligned with a synonym table (`lr` ↔ `learning_rate`), and values are compared after numeric normalisation ($3\text{e}{-4} = 0.0003$).

F. Chat, notation and context

Chat retrieves paragraph passages with BM25 [45] and local dense embeddings [46], [47], fused by reciprocal rank fusion [48]:

$$\mathrm{RRF}(d) = \sum_{r \in \{\text{bm25, dense}\}} \frac{1}{k + \mathrm{rank}_r(d)}, \qquad k = 60. \quad (3)$$

Passages are labelled [P1], [P2], … in the prompt. Python removes citation markers the model invents, refuses an answer with no citation, and refuses without an LLM call when nothing relevant is retrieved. For notation, the arXiv LaTeX source is read in memory (never compiled). Equations are numbered by LaTeX's own rules so that they match the PDF, and one call builds a glossary whose symbol meanings are each grounded in a sentence. For context, key references are the ones cited most often in the text, with the citing sentences as evidence. Semantic Scholar adds citation counts and influential citations [49] when it is reachable.

G. Review, report and comparison

Reviewer mode asks the LLM for a summary, strengths, weaknesses, missing ablations and questions for the authors, each point quoted. Python adds findings from the scorecard, the claims and the code check, and gives no score. A Word report and a multi-paper comparison matrix are built from stored results without an LLM call.

V. Experimental Setup

A. Gold set

The evaluation set contains ten widely cited ML papers spanning NLP and vision: Bahdanau et al. [50], VGG [51], ResNet [52], Transformer [53], BERT [54], RoBERTa [55], DistilBERT [56], SimCLR [57], ViT [58] and LoRA [59]. The gold set holds 469 items in seven fields: task, datasets, baselines, metrics, hyperparameters, compute and code URLs. An annotation guide defines what counts in each field.

Annotation provenance. Because of time constraints, the reference annotations were produced by an LLM (Claude) reading each paper's full text page by page under the guide. They were adjudicated once after the first comparison with PaperLens. Fourteen items that the papers state and that fit the definitions were added, one over-broad alias was removed, and nothing was removed to suit the system. Every change is logged. Our numbers therefore measure agreement with an *LLM-annotated, team-checked* reference, and we discuss this threat in Section VIII-A.

B. Matching and metrics

Values are normalised (Unicode NFKD, number words turned into digits, decimals kept). A predicted text item matches a gold item when the two are nearly equal (fuzzy ≥ 88 with equal digit sequences, so CIFAR-10 never matches CIFAR-100), or when one contains the other and the remainder is filler, or when the contained gold phrase is specific on its own. An item that packs several values is tried part by part. Hyperparameters must agree on the name (through a synonym table) and on the parsed value ($6\% = 0.06$). For a field with predicted items P and gold items G,

$$\mathrm{P} = \frac{|\{p \in P : \exists g,\, p \sim g\}|}{|P|}, \quad \mathrm{R} = \frac{|\{g \in G : \exists p,\, p \sim g\}|}{|G|}, \quad (4)$$

and F1 is their harmonic mean (micro over papers; macro = mean of per-paper F1). The *hallucination rate* is the share of extracted items that are unlocated, unsupported or without evidence. *Numeric accuracy* is the share of extracted hyperparameters, among those whose name is in the gold, that have a correct value. *Hyperparameter coverage* is the share of gold (paper, name) pairs for which a correct value was extracted. Checklist agreement uses accuracy and Cohen's κ [60].

TABLE III
Extraction results per field (10 papers, single run).

Field	Gold	Pred	P	R	F1	$\mathrm{F1}_{\mathrm{macro}}$
Task	31	14	0.79	0.32	0.46	0.52
Datasets	64	50	1.00	0.81	**0.90**	0.83
Baselines	87	70	0.99	0.72	0.83	0.83
Metrics	51	34	1.00	0.63	0.77	0.79
Hyperparameters	203	71	0.99	0.36	0.53	0.55
Compute	23	17	0.94	0.70	0.80	0.80
Code URLs	10	11	0.82	0.90	0.86	0.92
Core-field F1 (micro average of the five core fields)					**0.72**	
Hyperparameter coverage / numeric accuracy					43% / 100%	
Verified evidence / hallucination rate					80% / 10%	

C. Configuration

Extraction used Gemini Flash-Lite through the free tier (October 2026) with the section-group prompts, GROBID 0.8.1 and the default thresholds above. Reported numbers come from a single run (`phase2_flash_lite`). All tables and figures in this paper are regenerated by one command from the stored run.

VI. Results

A. Extraction quality

Table III and Fig. 2 report per-field scores. Precision is high throughout (0.79 to 1.00). When PaperLens reports a dataset, metric or baseline, it is almost always correct. Recall is the bottleneck, above all for hyperparameters (0.36) and tasks (0.32). The micro-averaged core-field F1 is 0.72, short of our target of 0.80. Every extracted hyperparameter whose name matched the gold carried the right value (numeric accuracy 100%), but only 43% of gold hyperparameters were covered.

Fig. 3 breaks F1 down by paper. Hyperparameter recall is lowest for papers that put their settings in appendix tables (LoRA: F1 0.06) or that list several model sizes (Transformer, BERT, DistilBERT). Task F1 is zero for DistilBERT and LoRA because their extracted tasks were stated more generally than the gold.

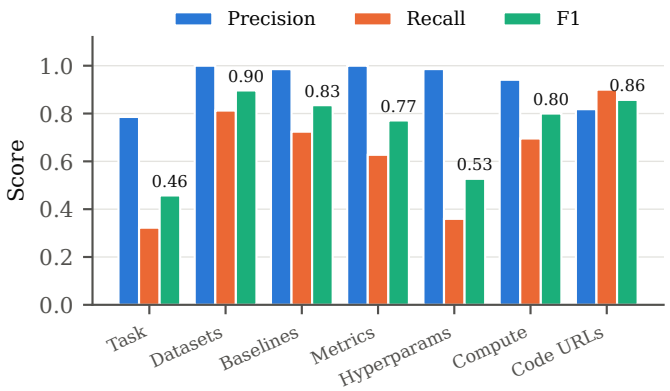


Fig. 2. Precision, recall and F1 per field. Precision is close to 1 everywhere, and recall limits F1.

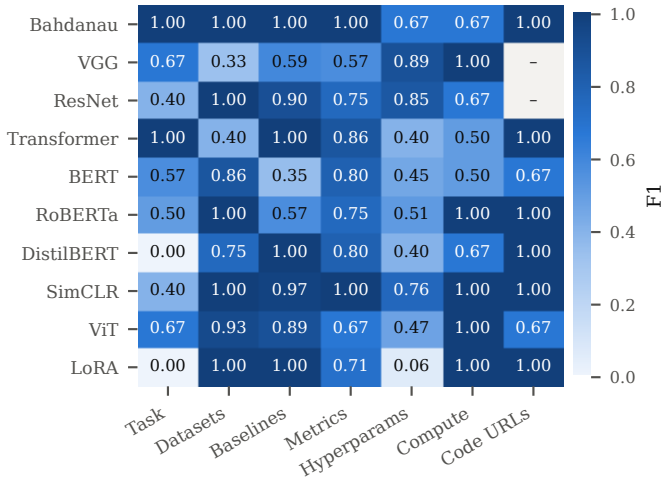


Fig. 3. Per-paper F1 by field ("–": no gold code URL for that paper). Hyperparameters in appendix tables (LoRA) are the main failure.

B. Evidence policy (ablation A3)

Because every item carries a status, we can score the same extraction under stricter evidence policies without new LLM calls (Fig. 4). Requiring the quote to be found in the paper lowers the hallucination rate from 10% to 2%. Additionally requiring the support check (supports or partly supports) lowers it to 0%, while precision stays at 0.96 to 0.97. The cost is recall, which falls from 0.54 to 0.43 under the strictest policy, because some correct items come with a paraphrased quote that cannot be located. We therefore keep all items in the interface and badge their status. The reader sees the evidence level, and the metrics still count every item.

C. Parser (ablation A1) and retrieval (ablation A6)

Fig. 5 compares GROBID with a PyMuPDF-only parse by the share of gold values still present in the parsed text, which is an upper bound on extraction recall. Raw PyMuPDF text keeps slightly more of the values (93% vs 89%), mostly numbers in tables and footnotes. GROBID, however, recovers the structure the rest of the pipeline depends on: 260 vs 181 section headings, 463 vs 344 references, 67 vs 30 equations, and in-text citations. With GROBID, 23% of gold hyperparameter values are absent from the parsed text, so no prompt change can recover them. For chat, BM25 alone finds the passage holding a gold answer in its top five for 47% of questions built from the gold set. The dense and hybrid arms are pending (Section VIII).

TABLE IV
ACCEPTANCE CHECKS OF THE REMAINING COMPONENTS.

Component	Criterion	Result
Parsing	10 arXiv papers; tables on ≥ 8	10/10
Extraction	full output ≤ 3 min	10/10; 15–50 s per paper
Repro. scorecard	$\geq 80\%$ agreement	75%, $\kappa = 0.57$ (below target)
Code discovery	$\geq 80\%$ repos found	8/8
Paper vs code	one real mismatch	BERT (below)
Chat / math / context	cited answers; glossary on 5 papers; graph	passed (VAE, Adam, DDPM, Transformer, LoRA)

D. Reproducibility, code and other checks

Table IV summarises the acceptance checks of the other components. The scorecard agreed with the team's annotations on 158 of 210 checklist decisions (75%, $\kappa = 0.57$). This is below the targets of 80% and $\kappa = 0.6$, but up from 65% before item-level guidance was added. The largest disagreements concern training time, re-run baselines and README instructions. Repository discovery found the official code for all eight papers that have it.

Case study: BERT. The paper states that pre-training used a learning rate of 10^{-4}, a batch size of 256 and 1M steps [54]. The static scan of the official repository found the defaults of `run_pretraining.py` to be 5×10^{-5}, 32 and 100,000. PaperLens reports this as a mismatch and shows the paper's sentence beside the code location. The difference is legitimate, since the script's values are command-line defaults, but anyone running the script as is would not reproduce the paper. This is exactly the gap the check is meant to surface.

The phase-5 check covered five math-heavy papers (VAE [61], Adam [62], DDPM [63], Transformer, LoRA). Chat answers cited pages, off-topic questions were refused, every glossary symbol's quote was verified, and the citation graph rendered.

E. Interface

Fig. 6 shows the workspace. Clicking an evidence chip scrolls the PDF to the quoted sentence and highlights it in the colour of its status. Fig. 8 shows the comparison matrix, built from the stored extractions of two papers without an LLM call. Each cell gives the page where the paper states the item.

VII. Error Analysis

The run missed 214 gold items and produced only 8 items that match nothing in the gold (Fig. 9). We observe five recurring causes:

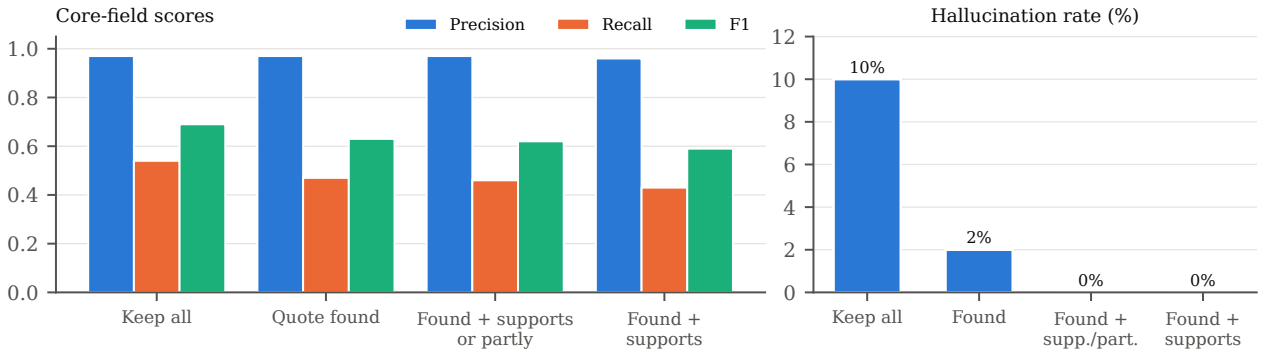


Fig. 4. Ablation A3: the same extraction under four evidence policies (core fields). Left: precision, recall and F1. Right: hallucination rate. Requiring a located, supporting quote removes hallucinated items at a moderate cost in recall.

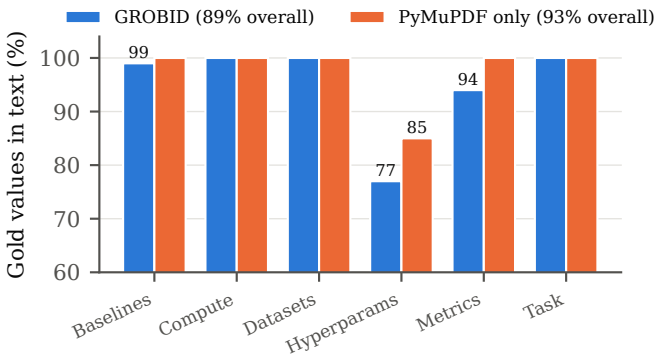


Fig. 5. Ablation A1: gold values still present in the parsed text, by field (unlabelled bars are 100%).

1) **Appendix and table hyperparameters (dominant).** Of the misses, 130 are hyperparameters. An example is Bahdanau et al.'s embedding size 620, maxout size 500 and gradient clipping at 1, all in the appendix. Asking for a quote per item seems to favour prose over tables, and long appendix groups dilute attention [24].
2) **Tasks listed too narrowly.** The model names one or two tasks, whereas the gold lists every task the experiments address (for example, VGG's localisation experiments).
3) **Baselines cited by author name only,** such as transfer-learning comparisons given as "Chatfield et al. 2014". Our scorer also undercounts one case: "Krizhevsky et al." does not match "Krizhevsky et al. 2012" because digit sequences must be equal.
4) **Wrong code link.** For BERT, a third-party library link was also reported. For ViT, a different repository's URL was reported, with the next word glued onto it by the PDF text layer.
5) **Parsing ceiling.** 11% of gold values do not survive GROBID's text at all (A1).

VIII. Discussion and Limitations

Precision by construction, recall by effort. Making the LLM quote its sources, and checking every quote in code, yields near-perfect precision and a measurable hallucination rate that can be driven to zero. Recall, however, depends on the model reading every table and appendix. Table-aware extraction of appendix settings is the most promising next step.

Deterministic decisions. Scores, deltas, verdicts and code comparisons are computed in Python. They are reproducible, testable with unit tests, and conservative by design: an ambiguous table match yields "unsupported", never "contradicted".

Limitations.

- The gold set is small: 10 papers instead of the planned 40, with no double annotation.
- The scorecard's agreement with the team is below target.
- Cloning repositories failed on the team's Windows machines, so the code comparison was demonstrated on Linux. Repository discovery itself needs no clone.
- Semantic Scholar often rate-limits, and the context view then falls back to a PDF-only citation graph.
- Several planned ablations are pending:
  - extraction granularity (A2);
  - rule hints (A4);
  - a stronger model (A5);
  - LLM vs Python claim verdicts (A7);
  - extraction without quotes (full A3);
  - dense and hybrid retrieval (A6);
  - LaTeX and Docling parsers (A1);
  - Qasper [26].

A. Threats to validity

The reference annotations were produced with an LLM and adjudicated once against this system's output. Where the annotating LLM and Gemini share blind spots, recall may be overestimated. Where the matcher is strict, as in the digit rule above, it may be underestimated. Adjudication only added items and never removed any to favour the system, and every change is logged. A human spot-check of two or three papers is in progress (TEAM: report the outcome). All results come from a single run of one model, the papers are well-known ML papers whose text the LLM may have seen in training, and absolute numbers may not transfer to recent or out-of-domain papers.

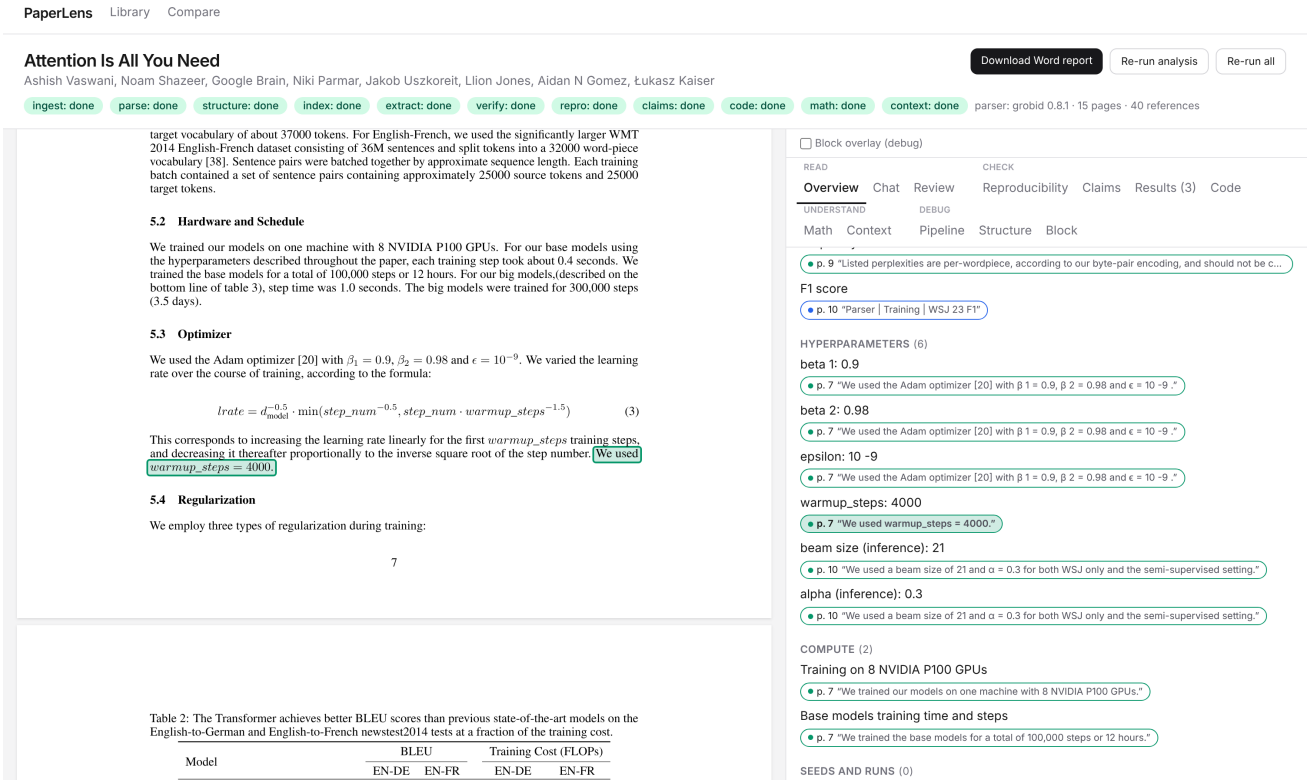


Fig. 6. The PaperLens workspace on the Transformer paper [53], with items from the evaluated Gemini run. Selecting the evidence for `warmup_steps: 4000` highlights the source sentence on p. 7 (left). Chip colours encode the evidence status (green = verified, blue = unchecked).

Table 2 The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fractio p. 8 CSV

Method	BLEU EN-DE ↑	BLEU EN-FR ↑	Training Cost (FLOPs) EN-DE ↓	Training Cost (FLOPs) EN-FR ↓
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		1.0 · 10^20
GNMT + RL [38]	24.6	39.92	2.3 · 10^19	1.4 · 10^20
ConvS2S [9]	25.16	40.46	9.6 · 10^18	1.5 · 10^20
MoE [32]	26.03	40.56	2.0 · 10^19	1.2 · 10^20
Deep-Att + PosUnk Ensemble [39]		40.4		8.0 · 10^20
GNMT + RL Ensemble [38]	26.30	41.16	1.8 · 10^20	1.1 · 10^21
ConvS2S Ensemble [9]	26.36	41.29	7.7 · 10^19	1.2 · 10^21
Transformer (base model)	27.3	38.1		3.3 · 10^18
Transformer (big)	28.4	41.8		2.3 · 10^19

Fig. 7. A results table rebuilt from word positions (Transformer, Table 2). The best value per column is computed in Python from the header's direction (higher BLEU and lower training cost are better).

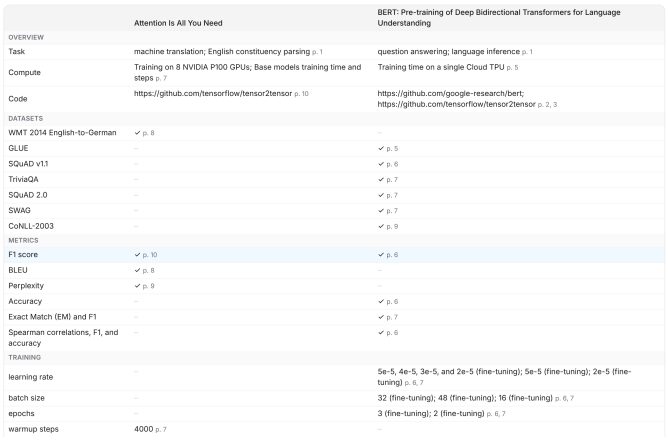


Fig. 8. Comparison matrix for two papers (excerpt): tasks, compute, code, datasets and metrics with the page each is stated on. Rows shared by both papers are tinted.

IX. Ethics and Privacy

Paper text is sent to a free-tier API whose inputs may be used to improve the provider's models. The interface warns users not to upload unpublished or confidential papers. Repository code is never executed. Reviewer mode gives no score, labels its output as an LLM draft, and separates it from facts computed in Python, consistent with evidence that LLM reviews should support human reviewers rather than replace them [43], [44].

X. Conclusion

PaperLens shows that a free-tier LLM can support careful paper reading when it is asked only to extract and quote, while deterministic code locates every quote and makes every decision. On ten papers it extracts with near-perfect precision and full numeric accuracy on matched hyperparameters. Every item is traceable to a highlighted sentence, and requiring evidence eliminates hallucinated items. Future work will target recall on appendix tables, complete the pending ablations, and extend the gold set with independent human annotation.

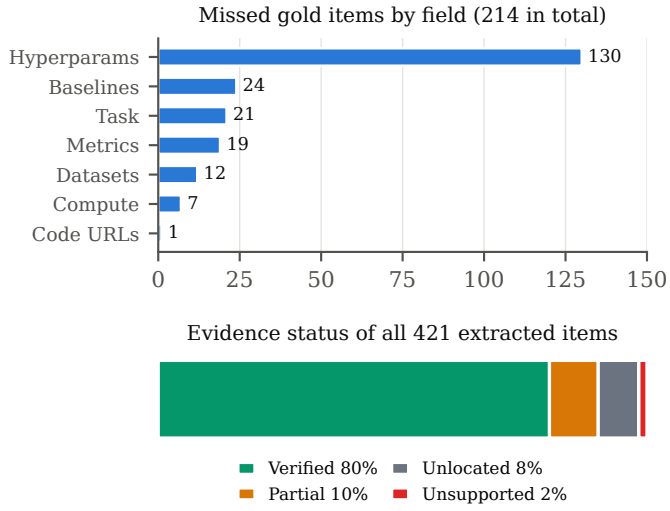


Fig. 9. Top: missed gold items by field. Bottom: evidence status of all extracted items.

Acknowledgment

The authors thank their project guide for feedback. PaperLens uses GROBID, PyMuPDF and the Gemini API free tier.

References

[1] O. E. Gundersen and S. Kjensmo, "State of the art: Reproducibility in artificial intelligence," in *Proc. AAAI Conf. Artificial Intelligence*, 2018, pp. 1644–1651.

[2] E. Raff, "A step toward quantifying independently reproducible machine learning research," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[3] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.

[4] C. Collberg and T. A. Proebsting, "Repeatability in computer systems research," *Communications of the ACM*, vol. 59, no. 3, 2016.

[5] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, 2023.

[6] H. Rashkin, V. Nikolaev, M. Lamm, L. Aroyo, M. Collins, D. Das, S. Petrov, G. S. Tomar, I. Turc, and D. Reitter, "Measuring attribution in natural language generation models," *Computational Linguistics*, vol. 49, no. 4, pp. 777–840, 2023.

[7] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 6465–6488.

[8] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," arXiv:2312.11805, 2023.

[9] Y. Luan, L. He, M. Ostendorf, and H. Hajishirzi, "Multi-task identification of entities, relations, and coreference for scientific knowledge graph construction," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

[10] S. Jain, M. van Zuylen, H. Hajishirzi, and I. Beltagy, "SciREX: A challenge dataset for document-level information extraction," in *Proc. Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2020.

[11] Y. Hou, C. Jochim, M. Gleize, F. Bonin, and D. Ganguly, "Identification of tasks, datasets, evaluation metrics, and numeric scores for scientific leaderboards construction," in *Proc. Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2019, pp. 5203–5213.

[12] M. Kardas, P. Czapla, P. Stenetorp, S. Ruder, S. Riedel, R. Taylor, and R. Stojnic, "AxCell: Automatic extraction of results from machine learning papers," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 8580–8594.

[13] I. Beltagy, K. Lo, and A. Cohan, "SciBERT: A pretrained language model for scientific text," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[14] J. Dagdelen, A. Dunn, S. Lee, N. Walker, A. S. Rosen, G. Ceder, K. A. Persson, and A. Jain, "Structured information extraction from scientific text with large language models," *Nature Communications*, vol. 15, p. 1418, 2024.

[15] P. Lopez, "GROBID: Combining automatic bibliographic data recognition and term extraction for scholarship publications," in *Research and Advanced Technology for Digital Libraries (ECDL)*, ser. Lecture Notes in Computer Science, vol. 5714, 2009.

[16] K. Lo, L. L. Wang, M. Neumann, R. Kinney, and D. S. Weld, "S2ORC: The semantic scholar open research corpus," in *Proc. Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2020.

[17] L. Blecher, G. Cucurull, T. Scialom, and R. Stojnic, "Nougat: Neural optical understanding for academic documents," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[18] C. Auer *et al.*, "Docling technical report," arXiv:2408.09869, 2024.

[19] X. Zhong, E. ShafieiBavani, and A. J. Yepes, "Image-based table recognition: Data, model, and evaluation," in *Proc. European Conf. Computer Vision (ECCV)*, 2020.

[20] B. Smock, R. Pesala, and R. Abraham, "PubTables-1M: Towards comprehensive table extraction from unstructured documents," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 4634–4642.

[21] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[22] S. Min, K. Krishna, X. Lyu, M. Lewis, W. tau Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[23] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[24] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, 2024.

[25] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[26] P. Dasigi, K. Lo, I. Beltagy, A. Cohan, N. A. Smith, and M. Gardner, "A dataset of information-seeking questions and answers anchored in research papers," in *Proc. Conf. North American Chapter Assoc. Computational Linguistics (NAACL-HLT)*, 2021.

[27] J. Lála *et al.*, "PaperQA: Retrieval-augmented generative agent for scientific research," arXiv:2312.07559, 2023.

[28] A. Asai *et al.*, "Synthesizing scientific literature with retrieval-augmented language models," *Nature*, 2026, doi:10.1038/s41586-025-10072-4.

[29] A. Head, K. Lo, D. Kang, R. Fok, S. Skjonsberg, D. S. Weld, and M. A. Hearst, "Augmenting scientific papers with just-in-time, position-sensitive definitions of terms and symbols," in *Proc. CHI Conf. Human Factors in Computing Systems*, 2021.

[30] D. Kang, A. Head, R. Sidhu, K. Lo, D. S. Weld, and M. A. Hearst, "Document-level definition detection in scholarly documents: Existing models, error analyses, and future directions," in *Proc. First Workshop on Scholarly Document Processing (SDP)*, 2020, pp. 196–206.

[31] K. Lo, J. C. Chang, A. Head, J. Bragg, A. X. Zhang, C. Trier *et al.*, "The semantic reader project: Augmenting scholarly documents through AI-powered interactive reading interfaces," *Communications of the ACM*, vol. 67, no. 10, 2024.

[32] I. Cachola, K. Lo, A. Cohan, and D. S. Weld, "TLDR: Extreme summarization of scientific documents," in *Findings of the Assoc. Computational Linguistics: EMNLP*, 2020.

[33] D. Wadden, S. Lin, K. Lo, L. L. Wang, M. van Zuylen, A. Cohan, and H. Hajishirzi, "Fact or fiction: Verifying scientific claims," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[34] W. Chen, H. Wang, J. Chen, Y. Zhang, H. Wang, S. Li, X. Zhou, and W. Y. Wang, "TabFact: A large-scale dataset for table-based fact verification," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2020.

[35] X. Lu, L. Pan, Q. Liu, P. Nakov, and M.-Y. Kan, "SCITAB: A challenging benchmark for compositional reasoning and claim verification on scientific tables," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 7787–7813.

[36] J. Dodge, S. Gururangan, D. Card, R. Schwartz, and N. A. Smith, "Show your work: Improved reporting of experimental results," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[37] X. Bouthillier, C. Laurent, and P. Vincent, "Unreproducible research is reproducible," in *Proc. Int. Conf. Machine Learning (ICML)*, 2019.

[38] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023.

[39] B. Bogin, K. Yang, S. Gupta, K. Richardson, E. Bransom, P. Clark, A. Sabharwal, and T. Khot, "SUPER: Evaluating agents on setting up and executing tasks from research repositories," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 12 622–12 645.

[40] Z. S. Siegel, S. Kapoor, N. Nadgir, B. Stroebl, and A. Narayanan, "CORE-Bench: Fostering the credibility of published research through a computational reproducibility agent benchmark," *Transactions on Machine Learning Research*, 2024.

[41] G. Starace *et al.*, "PaperBench: Evaluating AI's ability to replicate AI research," in *Proc. Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 267, 2025.

[42] W. Yuan, P. Liu, and G. Neubig, "Can we automate scientific reviewing?" *Journal of Artificial Intelligence Research*, vol. 75, pp. 171–212, 2022.

[43] W. Liang, Y. Zhang *et al.*, "Can large language models provide useful feedback on research papers? a large-scale empirical analysis," *NEJM AI*, vol. 1, no. 8, 2024.

[44] R. Zhou, L. Chen, and K. Yu, "Is LLM a reliable reviewer? a comprehensive evaluation of LLM on automatic paper reviewing tasks," in *Proc. Joint Int. Conf. Computational Linguistics, Language Resources and Evaluation (LREC-COLING)*, 2024, pp. 9340–9351.

[45] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[46] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[47] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[48] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2009.

[49] M. Valenzuela, V. Ha, and O. Etzioni, "Identifying meaningful citations," in *AAAI Workshop on Scholarly Big Data: AI Perspectives, Challenges, and Ideas*, 2015.

[50] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[51] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[52] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.

[53] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[54] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North American Chapter Assoc. Computational Linguistics (NAACL-HLT)*, 2019.

[55] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[56] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.

[57] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Machine Learning (ICML)*, 2020.

[58] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

[59] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[60] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[61] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2014.

[62] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[63] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.